TraceHG: An Unsupervised Dual-View Framework for Microservice Anomaly Detection

Ningning Han, *Member, IEEE*, Siyang Lu, *Member, IEEE*, Zaichao Lin, *Graduate Student Member, IEEE*, Bin Li, *Member, IEEE*, Nan Wang, *Graduate Student Member, IEEE*, and Xin Luo, *Fellow, IEEE*

***Abstract*—Traces and logs record the behaviors and interactions between microservices, making them indispensable for diagnosing system anomalies. However, the intricate and flexible architecture of microservices presents significant challenges for automated anomaly detection. Existing approaches often struggle to capture the internal dynamics and correlations among microservices, which limits their ability to comprehensively detect anomalies arising from synchronous or asynchronous calls. Therefore, we categorize three prevalent anomaly paradigms in microservice systems: intra-service, inter-service, and joint anomalies. To address these challenges, we propose a novel unsupervised learning framework for dual-view anomaly detection, named TraceHG. Specifically, it constructs the trace graph composed of logs, response times, and traces. Besides, TraceHG utilizes hypergraph transformation to establish two views, providing comprehensive information in microservices. Furthermore, we introduce a dual-view framework that leverages both a hypergraph-view and a graph-view encoders to explicitly learn associations among microservices. These two views, employing GCN and HGNNs, capture representations of internal dynamics, invocations, and their associated contexts in microservices. By constructing two minimized hyperspheres and measuring the distances from their centers in the latent space, we identify anomalies based on the anomaly scores. Extensive experiments on public benchmark datasets demonstrate that the proposed framework outperforms several state-of-the-art baselines in microservice anomaly detection.**

***Index Terms*—Microservice, Big Data, anomaly detection, graph neural networks.**

I. Introduction

MICROSERVICE architectures typically follow a distributed design, decomposing applications into multiple independent services, each executing within its own process or container [1], [2], [3]. Due to its efficient resource allocation and flexible deployment, it is widely appealing to industrial and enterprise applications. However, substantial challenges have influence on system reliability because of its intricate and dynamic constructions. Specifically, microservices are prone to failure from various sources, such as hardware malfunctions, misconfigurations, implementation errors, and faulty coordination in service interactions. Moreover, the uncertain environment created by issues like data leakage and unauthorized access further complicates system maintenance. In light of these challenges, logs play a critical role, as they are widely used to record the behaviors of each service. Simultaneously, traces describe the invocations between microservices, providing insight into these systemic challenges. A trace represents the complete execution path of a user request in a distributed system, spanning multiple services or components. It records the full invocation chain corresponding to a single request. Within each trace, the system generates logs that capture fine-grained runtime events during service execution. Fig. 1. illustrates the relationship between trace, and log. As presented in the existing work [4], [5], [6], trace and log-based anomaly detection methods are tailored for automatically identifying and mining potential issues within distributed systems.

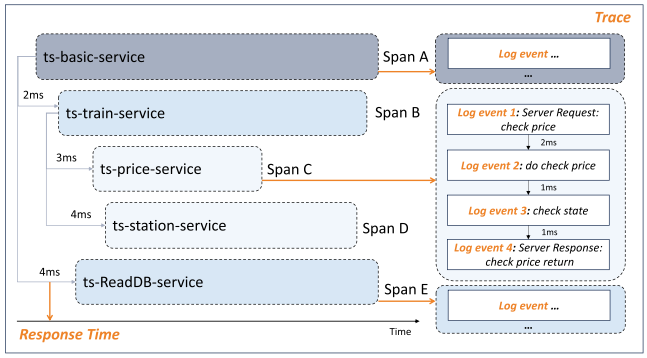

Fig. 1. Relationship between trace, log, and response time. A trace consists of multiple spans, each containing multiple logs. Response time is defined as the time interval between consecutive log events, capturing fine-grained runtime dynamics.

Recent research [7], [8], [9], [10] leverage deep learning-based trace and log analysis methods to automatically detect runtime anomalies of microservice systems. Owing to the unique invocation relationships in microservices, many researchers [6], [11], [12], [13], [14] have utilized Graph Neural Networks (GNNs) and their variants to capture the topology structure among traces. By leveraging graph-based message passing,

Received 3 November 2024; revised 15 January 2026; accepted 18 February 2026. Date of publication 24 February 2026; date of current version 10 April 2026. This work was supported in part by the General Program of National Natural Science Foundation of China under Grant 62376023 and in part by the State Key Laboratory of Internet Architecture, Tsinghua University, under Grant HLW2025MS03. *(Corresponding author: Siyang Lu.)*

Ningning Han, Siyang Lu, Zaichao Lin, and Bin Li are with the School of Computer Science and Technology, Beijing Jiaotong University, Beijing 100044, China (e-mail: 22120493@bjtu.edu.cn; sylu@bjtu.edu.cn; lzc_eastchina@bjtu.edu.cn; 24120434@bjtu.edu.cn).

Nan Wang is with the School of Cyberspace Science and Techonology, Beijing Jiaotong University, Beijing 100044, China (e-mail: wangnanbjtu@bjtu.edu.cn).

Xin Luo is with the College of Computer and Information Science, Southwest University, Chongqing 400715, China (e-mail: luoxin@swu.edu.cn).

Digital Object Identifier 10.1109/TSC.2026.3667576

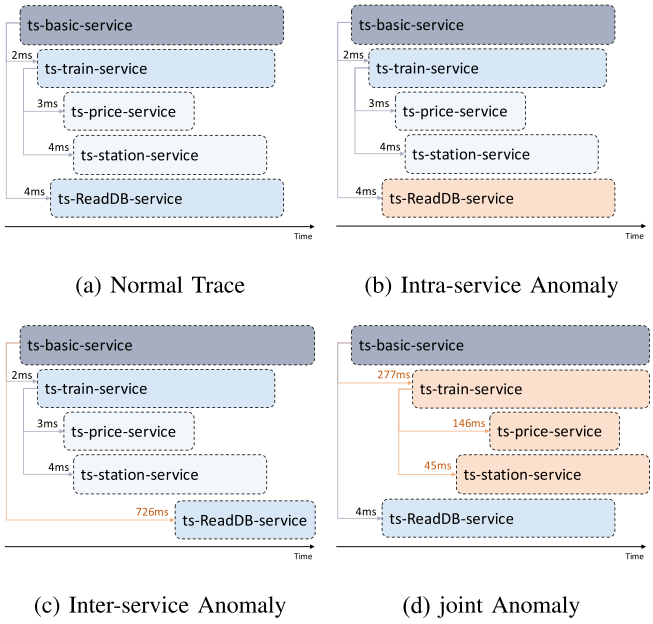

Fig. 2. Illustration of a normal trace and three typical anomalies in the microservice framework. Arrows represent invocations between microservices, with orange indicating abnormal invocations, whereas black represents normal ones. The numbers above the arrows represent the microservice response time.

these models effectively capture latent relationshipes in invocation chains of microservices.

Despite the success of existing trace and log-based microservice anomaly methods they still suffer from a significant limitation. Specifically, the significance of response time as a manifestation of microservice behavior has been overlooked, which hinders comprehensively detecting anomalies. To describe system performance at a more granular level, we define the response time as the time interval between every two consecutive log events. In practice, anomalies in microservice systems often manifest as abnormal patterns in response time sequences. As shown in Fig. 2, there are three commonly observed types of anomalies, each exhibiting distinct temporal characteristics. The following summarizes their representative behaviors:

- *Intra-service anomaly:* intra service anomalies are typically caused by issues internal to a microservice, such as a corrupted cache or file I/O errors, and are generally unrelated to interactions between services [15]. These anomalies usually do not have an immediate impact on the response time of the overall invocation chain. However, they can still result in localized latency spikes or irregular timing patterns within the affected service. If such anomalies occur frequently or persist under high load, they may eventually affect downstream services, thereby evolving into inter-service anomalies and leading to broader performance degradation.
- *Inter-service anomaly:* The inter-service anomalies stem from the environmental configurations under which microservices are deployed [16]. For instance, other processes running in the same environment as microservices occupy memory or CPU, resulting in the microservices being unconditional. These anomalies arise from issues in the deployment environment, such as CPU or memory contention caused by co-located processes or container resource limits. Since these factors impact the runtime performance of services, they usually result in gradual or persistent increases in response time for specific services or invocation paths.
- *Joint anomaly:* The joint anomaly can be triggered when strong dependencies between microservices, long invocation chains, or a critical service fails [17], [18]. It can cause a cascading reaction across multiple distributed nodes. Unlike intra- or inter-service anomalies that are localized, joint anomalies often manifest as a series of abnormal response times across multiple services along the invocation path. For instance, if a payment service slows down, upstream services like order processing, inventory, and even user-facing APIs may not immediately fail, but will experience progressively increasing delays. This latency propagation leads to queuing, retries, and eventually system-wide degradation.

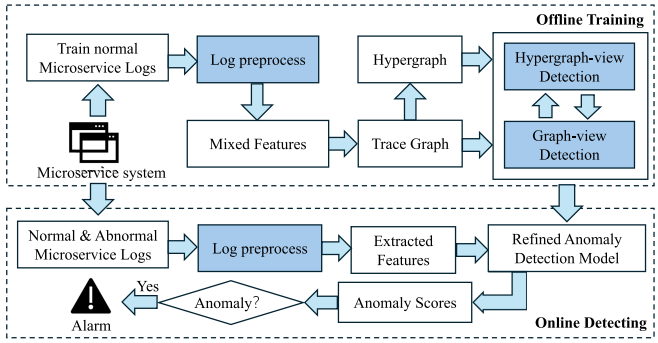

Fig. 3. Overview of TraceHG. The proposed framework consists of two main stages: offline training and online detection. The offline training phase utilizes only normal microservice logs to learn the normal behavior patterns, while the online detection phase applies the trained model to incoming traces for real-time anomaly detection.

These three scenarios demonstrate that response time is a key behavioral signal that reflects both local processing conditions and the dynamic state of microservice interactions. However, many existing trace-based approaches overlook response time or capture the response time implicitly, leading to the omission of anomalies, particularly in the case of inter-service and joint anomalies.

To address this issue, we propose a novel unsupervised learning framework that explicitly incorporates response time as a core component. Our approach is designed to capture fine-grained spatiotemporal patterns from traces and logs, introducing response time and explicitly modeling spatiotemporal features, to improve the detection of complex anomalies, especially, inter-service and joint anomalies in microservices. As illustrated in Fig. 3, the proposed framework is divided into offline training and online detecting. During offline training, normal microservice logs are used for model training. After training, a refined anomaly detection model is employed to detect microservice anomalies. To construct TraceHG, we first reorganize the recorded microservice events into traces in

chronological order. Each trace is then transformed into a graph. In parallel, the log events are parsed and embedded into the graph structure. Besides, the hypergraph transformed from the original trace graph is designed to explicitly represent response time and interaction information among microservices. Moreover, we design two views: a graph view and a hypergraph view to learn temporal and spatial information from the trace graph and the hypergraph. The trace graph and hypergraph are employed as inputs for the graph-view encoder and hypergraph-view encoder, respectively. We facilitate mutual detection by swapping the surrounding contexts of graph and hypergraph views. Subsequently, two minimized hyperspheres are generated by graph and hypergraph views, respectively. Finally, by combining both graph-view and hypergraph anomaly scores calculated from hyperspheres, we obtain a comprehensive objective to detect complex microservice anomalies thoroughly. In summary, the main contributions of the proposed work are as follows:

- We propose an unsupervised learning model based on dual graph and hypergraph views, capturing the correlations and internal dynamics in microservices to enhance the capability of comprehensively detecting anomalies.
- We additionally incorporate temporal information to assist in the construction of the hypergraph, explicitly detecting inter-service and joint anomalies that are related to response time.
- Extensive experiments on the public benchmark datasets show that TraceHG achieves superior performance on anomaly detection compared with state-of-the-art (SOTA) baselines.

The remainder of the paper is organized as follows. In Section II, we survey recent related work. Section III proposes our approach. Section IV shows the experiment setup and experimental results. Section V discusses the comparison between single-view and dual-view approaches, as well as different log processing methods. Section VI summarizes this paper and presents our future work.

II. Related Work

In this section, we provide an overview of log anomaly detection and trace anomaly detection, with a focus on methods based on GNNs. We will also briefly introduce several widely used approaches for anomaly detection within each category.

A. Unsupervised Log-Oriented Anomaly Detection Methods

Traditional anomaly detection approaches for systems and applications are mainly based on log analysis. When a machine learning or statistical model [19] on the normal state of the system, deviations from established behavioral patterns in new logs are identified as anomalies. Farzad et al. [20] employed radius-based fuzzy C-means and a multilayer perceptron (MLP) network to detect suspect logs. Besides, Ying et al. [21] improved the KNN algorithm using average weighting technology, achieving higher accuracy on unbalanced log data.

More recently, unsupervised deep learning techniques have become prevalent due to the scarcity of labeled anomaly data and the complexity of log patterns [22], [23]. For instance, DeepLog [5] employs LSTM networks trained exclusively on normal log sequences to detect deviations in real-time. LogAnomaly [8] further integrates semantic features such as synonyms and antonyms, improving the representation of normal behaviors without requiring anomaly labels. Similarly, LogBERT [24] utilizes transformer-based architectures with self-supervised learning tasks to model normal log sequence distributions. Besides, Li et al. [25] proposed an unsupervised multi-parameter log anomaly detection method to address the challenge of dataset diversity. Zhang et al. [26] introduced a multivariate log-based anomaly detection approach tailored for distributed database systems and released a new dataset specifically designed for such environments. Nevertheless, anomaly detection approaches that rely solely on sequence or semantic representations of log events are inadequate for meeting the specific and complex architectural requirements of microservice anomaly detection systems.

B. Trace-Based Anomaly Detection Methods in Microservices

Traces record the execution paths, dependency relationships, and interactions among different services, offering a rich source of information for microservice anomaly detection [27], [28], [29], [30]. Given the complex and dynamic architecture of microservices, trace-based methods have become increasingly popular. In particular, unsupervised trace anomaly detection methods have shown great promise, as they avoid the need for labeled data while effectively modeling intricate service interactions. For example, Meng et al. [31] proposed a structure-based detection method that computes anomaly scores using tree edit distances between execution traces and identifies faulty components by analyzing structural differences. Similarly, Li et al. [32] designed a dynamic sliding window mechanism and a threshold-based decision boundary to detect anomalies based on call tracking information.

Recent methods further incorporate GNNs to model both topological and temporal features of traces. TraceGra [14] adopts an unsupervised encoder-decoder framework combining GNNs and LSTM networks to extract both structural and sequential patterns. Likewise, BSDG [33] leverages a dual-GCN architecture with mutual attention mechanisms to learn effective representations from attribute dependency graphs. Other approaches explore advanced learning strategies. For instance, UAC-AD [34] introduces adversarial contrastive learning to enhance representation learning on hard-to-distinguish samples. Both Li et al. [35] and Wang et al. [36] incorporate contrastive learning and utilize multi-modal inputs, including logs and KPIs, to improve anomaly detection performance in microservice environments. Overall, unsupervised and graph-based methods have emerged as effective solutions for microservice anomaly detection, as they can robustly handle unlabeled data and capture the structural complexity of modern service interactions.

III. Methodology

In this section, we introduce the proposed method TraceHG, which comprises three modules: *Log and Trace preprocessing*, *Model Training* and *Anomaly Detection*. The fine-grained

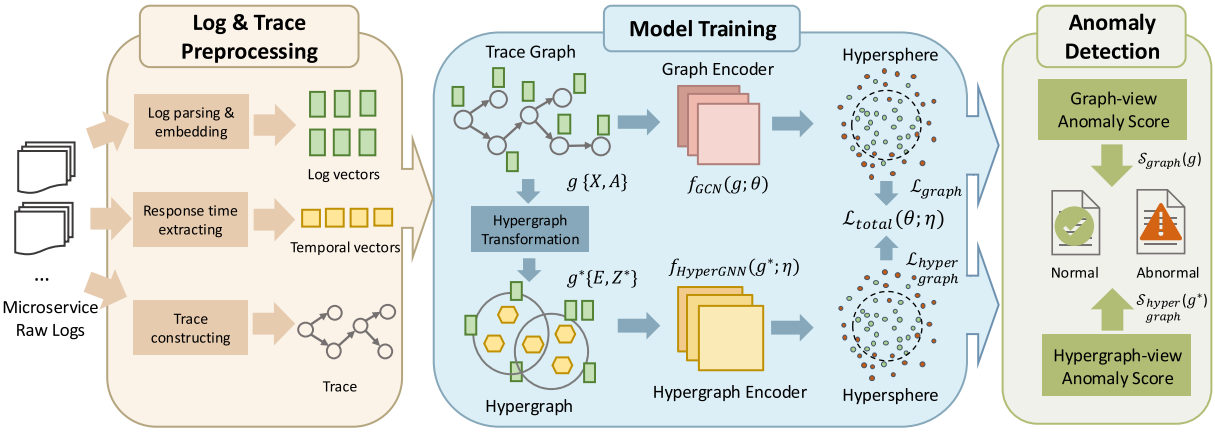

Fig. 4. Detailed architecture of TraceHG. The model operates in three stages: 1) Log and trace parsing, which includes log parsing and embedding, response time extraction, and trace construction; 2) Model training, where the graph and hypergraph views are encoded using specialized modules; and 3) Anomaly detection, in which anomaly scores from both views are combined to detect anomalous traces.

pipeline is shown in Fig. 4. In the Log and Trace parse module, we preprocess the raw log sequences and response time into log vectors and temporal vectors, separately. Simultaneously, intricate relationships in microservices are extracted into trace graphs from log events. After that, in the Model Train module, an unsupervised dual-view framework is used to learn potential representations. Lastly, anomaly scores generated from the dual-view framework determine whether the microservice logs are abnormal or not. The following subsections detail the log and trace preprocessing, model training, and anomaly detection.

A. Log and Trace Preprocessing

In this subsection, we aim to reserve as much latent information as possible from microservice logs by constructing a trace graph that integrates monolithic log data, response times, and invocation relationships. The trace graph is composed of log vectors, temporal vectors, and traces, which are generated through log parsing and embedding, response time extraction, and trace construction, respectively. Subsequently, the trace graph is then input into the anomaly detection model.

1) Log Parsing and Embedding: To convert raw logs into structured data, TraceHG employs the widely-used log parsing method called Drain [37]. This method extracts log templates from log messages. Log messages typically consist of both constant parts, known as log templates, and variable parts, which include elements like IP addresses and file IDs. We retain the log templates while filtering out the variable parts. For example, the log message: `cancel.service.CancelServiceImpl-[Cancel Order]calculate refund - 18.00` can be resolved to `cancel.service.CancelServiceImpl -[Cancel Order]calculate refund - <*>`, where `18.00` is filtered.

After log parsing, each log message is mapped to a log template. To semantic information as much as possible, log templates are embedded using a state-of-the-art pre-trained embedding algorithm named GloVe [38]. GloVe extracts semantic information from each word within a processed log event into a d-dimensional vector, where d is 300 in GloVe word vectors. Notably, there are composite tokens (e.g. `FrameworkServlet`) that must be decomposed into individual words (e.g., `Framework Servlet`) before processing. Simultaneously, non-character tokens such as delimiters or operators are dropped. After each word is transformed into a d-dimensional vector through word embedding, TraceHG converts a log event into a semantic vector by aggregating all word vectors within the log event. The proposed approach employs TF-IDF [39], a well-established method in information retrieval, for aggregation, taking into account the significance of each word. TF-IDF effectively measures the importance of words within the log event, allowing for an understanding of the event's semantic content during aggregation. TF (Term Frequency) quantifies the frequency of occurrence of a word. Meanwhile, IDF (Inverse Document Frequency) gauges the commonality or rarity of a word across all log events. The weight of each word in the log event can be produced by $TF \times IDF$. Subsequently, the log vector of a log event can be computed by multiplying the average of all words in the log event with their TF-IDF weights. By log parsing and embedding, we standardize log data, enabling subsequent analysis and anomaly detection.

2) Response Time Extracting: Each log timestamp records when the log event occurred. To boost the proficiency of detecting inter-service anomalies, TraceHG extracts the time interval between two corresponding log events. Next, the time spans are combined by trace ID. Moreover, the significant variance in time across different traces [40] (ranging from dozens to thousands of milliseconds) complicates the convergence of corresponding weights, thereby posing challenges in model training. Therefore, significant differences between time intervals are accounted for in the proposed approach by normalizing the extracted time vectors. An interval between log events can be denoted by Δt. Furthermore, the time vector of g-th trace graph is suggested by $T_g = [\Delta t_1, \Delta t_2 \ldots \Delta t_m]$, where m is the length of the trace k. Also, each time vector is standardized for serving as the response temporal weight W_T of hyperedge.

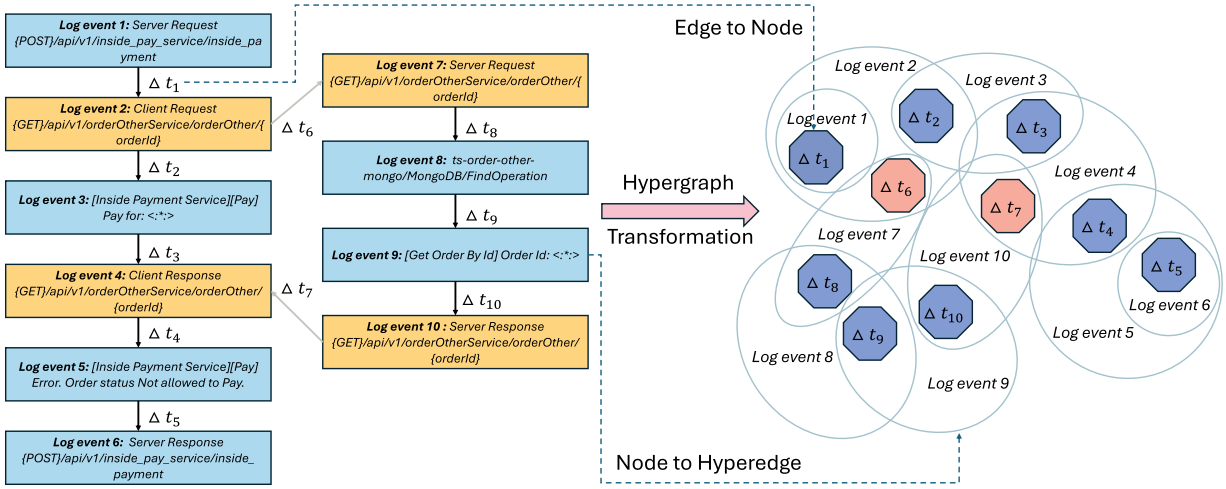

Fig. 5. The schematic diagram of hypergraph transformation. The left part represents a graph, and the right part represents a hypergraph. The yellow log event in the graph denotes an invocation between two microservices which correspond to the flesh pink hexagon in the hypergraph.

3) Trace Graph Constructing: Microservice traces, represented as spans, typically exhibit a complex structure characterized by a hierarchy of service invocations. TraceHG extracts these microservice invocations from log events. There are four key relationships between these log events: sequential execution within the microservice, synchronous calls, asynchronous calls, and asynchronous responses [13].

Firstly, to organize sequential execution, TraceHG gathers all logs belonging to a trace and arranges them in chronological order. Secondly, TraceHG inserts and organizes the entire trace graph based on the type of span. For each span, we establish its linkage with the parent span based on the communication pattern. In the case of a Client/Server interaction, we construct a synchronous request edge from the client-side request event in the parent span to the corresponding server-side request event in the current span. Similarly, a synchronous response edge is added from the server-side response event in the current span to the client-side response event in the parent span. For Producer/Consumer spans, we define an asynchronous edge from the producer event of the parent span to the consumer event in the current span.

Lastly, we construct the trace graph by treating each log event as a node. Directed edges are established between nodes based on their invocation relationships, with the corresponding response time used to define the edge weight. The edge attributes indicate the type of interaction, including synchronous calls, asynchronous messages, or sequential execution. The processed response time vector is thus naturally integrated into the graph structure through edge weights and serves as a key feature for hypergraph construction.

B. Model Training

In this subsection, a dual-view anomaly detection approach is employed to enhance the comprehensiveness of microservice anomaly detection. This phase begins by transforming the processed trace graph into a hypergraph. Additionally, we introduce both graph-view and hypergraph-view encoders to map the graph and hypergraph into hidden space. By leveraging both view structures, TraceHG captures complementary relational information within microservice traces. Finally, the one-class SVDD model is employed as the loss function for both views, optimizing the encoders to confine normal patterns within hyperspheres in latent space.

1) Hypergraph Transformation: In order to explicitly learn the edge information in the log graph, the proposed model adopts hypergraph transformation. Inspired by edge representation learning and message-passing mechanism [41], [42], the hypergraph transformation constructs a hypergraph based on the original graph. Given the paramount importance of accurately acquiring information on edges, we adopt a meticulous approach by explicitly employing message passing across edges to refine edge representations optimally. The schematic diagram is denoted as Fig. 5. It demonstrates the process of converting a normal trace graph into a hypergraph.

The original trace graph can be denoted by $\mathcal{G} = \{X, A\}$, where $X \in \mathbb{R}^{n \times d}$ is the embedding feature matrix of each trace graph, $A \in \mathbb{R}^{n \times n}$ is the adjacency matrix of $\mathcal{G}$. According to hypergraph duality and the concept of edge representation [42] within graphs, the edges and nodes of the original trace graph are transformed into the nodes and hyperedges of a hypergraph. Given the origin graph $\mathcal{G} = \{X, A\}$, the hypergraph $\mathcal{G}^* = \{E, Z^*\}$ is transformed by converting graph edges and nodes. Nodes embedding feature matrix in the hypergraph can be represented by:

$$E[k,:] = W_T \cdot X^* = W_T \cdot \frac{1}{2}\left(X[i,:] + X[j,:]\right), \qquad (1)$$

where $e_k = (v_i, v_j)$ and W_T is the temporal interval weight.

Besides, $Z \in \mathbb{R}^{n \times m}$ is the incidence matrix of the origin trace graph that represents the relationship between each node and edge. $Z^* = Z^T \in \mathbb{R}^{m \times n}$ is defined as the hypergraph incidence matrix. Since the structural and attribute information of the

trace graph is equivalent to the derived hypergraph, we introduce a simple yet effective augmentation during the hypergraph transformation process. TraceHG drops unnecessary edges from the hypergraph. To mitigate the risk of introducing additional anomalous nodes or edges, we randomly remove nodes from the hyperedges according to an i.i.d. Bernoulli distribution. It is worth noting that the partial disruption of high-order relations does not significantly affect the semantics of hyperedge representations. On the contrary, it can provide abundant representations of the vanilla trace graph.

2) Graph-View Encoder: Graph view and hypergraph view are used to explicitly learn the representations of nodes and edges in the trace graph, respectively. Therefore, n (node) and e (edge) are employed as subscripts for these two perspectives of representation. Given the capability of extracting latent features from trace log events, Graph Convolutional Network (GCN) [43] is adopted as the off-the-shelf graph encoder in graph view. The graph-view encoder takes the graph $\mathcal{G}$ as input, and outputs the graph-level context embedding $\hat{h}_n$:

$$\hat{h}_n = f_{GCN}(X, A), \tag{2}$$

where A represents the adjacency matrix of the trace graph $\mathcal{G}$. In the GCN model, the representation of nodes $h_n^{(l)}$ is computed as follows:

$$h_n{}^{(l)} = \sigma\left(D_n^{-\frac{1}{2}}\tilde{A}D_n^{-\frac{1}{2}}h_n^{(l-1)}\theta^{(l)}\right), \tag{3}$$

$$h_n{}^{(0)} = X, \tag{4}$$

where $\sigma(\cdot)$ is nonlinear activation function and the $PReLU$ is adopted here; $\theta \in \mathbb{R}^{d\times d'}$ is learnable parameters of the $l\text{-}th$ hidden layer; $h_n^{(l)}$ denotes the representation of the $l\text{-}th$ hidden layer. Specifically, $h_n^{(0)}$ is assigned by the initial feature embedding matrix X. D_n denotes the diagonal node degree matrix as follows:

$$D_n(i,i) = \sum_j \tilde{A}(i,j), \tag{5}$$

where $\tilde{A}$ is the self-connected adjacency matrix suggested by:

$$\tilde{A} = A + I. \tag{6}$$

The representation $h_n^{(L)}$ from the L-th layer serves as the representation of the final node in the graph-view encoder. The graph-level representation $\hat{h}_n$ of the trace graph can be derived utilizing an average readout function:

$$\hat{h}_n = Readout\left(h_n^{(L)}\right) = \frac{1}{N_g}\sum_{i=1}^{N_g} h_n^{(L)}[i,:], \tag{7}$$

where N_g represents the number of nodes in trace graph $\mathcal{G}$.

3) Hypergraph-View Encoder: Following the hypergraph transformation, the hypergraph is obtained. A hypergraph consists of a set of hyperedges and nodes, which are respectively transformed from the nodes and edges of the original trace graph. Similarly, we adopt the hypergraph-view encoder to map features of the hypergraph into the hidden space. The two-layer hypergraph graph neural network (HyperGNN) [44] is employed as the hypergraph-view encoder network. Hypergraph $\mathcal{G}^* = \{E, Z^*\}$ is fed into the HGNN, generating the hypergraph-level representation $\hat{h}_e$, which is defined as follows:

$$\hat{h}_e = f_{HyperGNN}(E, Z^*), \tag{8}$$

$$h_e{}^{(l)} = \sigma\left(D_v^{-\frac{1}{2}}Z^*W_eD_h^{-1}Z^{*T}D_v^{-\frac{1}{2}}h_e^{(l-1)}\eta^{(l)}\right), \tag{9}$$

where $h_e^{(l)}$ denotes the $l\text{-}th$ hidden layer representation. When $l = 0$, $h_e^{(0)} = E$. And, $\sigma(\cdot)$ is a nonlinear activation function and the $Tanh$ is adopted here. $\eta^{(l)} \in \mathbb{R}^{d\times d'}$ denotes learnable parameters of HyperGNN. The weight matrix of hyperedges, denoted as W_e, is defined as the identity matrix.

The diagonal node degree matrix D_v and hyperedge degree matrix D_h of Z^* are represented as follow:

$$D_v(i,i) = \sum_j Z^*(i,j), \tag{10}$$

$$D_h(j,j) = \sum_i Z^*(i,j). \tag{11}$$

The representation $h_e^{(L)}$ is obtained from the hypergraph-view encoder. Furthermore, the hypergraph-level representation $\hat{h}_e$ is generated from the average readout function:

$$\hat{h}_e = Readout\left(h_e^{(L)}\right) = \frac{1}{N_h}\sum_{i=1}^{N_h} h_e^{(L)}[i,:] \tag{12}$$

where N_h represents the number of nodes in hypergraph $\mathcal{G}^*$.

4) Loss Function: The behavior of normal trace events consistently exhibits similarity, whereas abnormal trace events show greater variability. Therefore, One-class SVDD [45] is employed to train a minimized hypersphere using normal embedding vectors, which serve as the loss function. Both the graph view and hypergraph view utilize the minimized hypersphere. The graph loss function is defined as follows:

$$\mathcal{L}_{graph} = \frac{1}{\mu_g N}\sum_{n=1}^{N}\max\left\{||\hat{h}_n - c_g|| - R_g^2, 0\right\} + R_g^2 + \frac{1}{\lambda}\sum_{l=1}^{L}||\theta^{(l)}||_F^2, \tag{13}$$

where c_g is the center of the graph-view hypersphere; N is the number of trace graphs; L is the number of layers in the graph-view encoder, 2 is adopted here; R_g is the radius of the hypersphere. It is important to note that frequent updates to the centers of the hypersphere have negligible effects, and as a result, the centers are not updated after the initial epochs. $\frac{1}{\lambda}\sum_{l=1}^{L}||\theta^{(l)}||_F^2$ is a weight decay regularizer of graph-view parameters. Additionally, μ_g is a hyperparameter that balances the volume of the hypersphere and also controls the updates to R_g. In TraceHG, R_g is updated only after the warming epochs. We train the hidden representations of normal trace graphs to be as compact as possible, in order to distinguish them from abnormal trace graph representations. Similarly, the hypergraph view also defines a hypersphere. The hypergraph-view loss function, $\mathcal{L}_{hypergraph}$, is analogous to that of the graph view.

TABLE I
SUMMARY OF NOTATIONS

Notations	Definitions
$\mathcal{G} = \{X, A\}$	A directed attributed trace graph
$\mathcal{G}^* = \{E, Z^*\}$	A transformed hypergraph
$X \in \mathbb{R}^{n \times d}$	The node feature matrix of the trace graph $\mathcal{G}$
$Z \in \mathbb{R}^{n \times m}$	The incidence matrix of the trace graph $\mathcal{G}$
$A \in \mathbb{R}^{n \times n}$	The adjacency matrix of the trace graph $\mathcal{G}$
$E \in \mathbb{R}^{m \times d}$	The node feature matrix of the hypergraph $\mathcal{G}^*$
$Z^* = Z^T \in \mathbb{R}^{m \times n}$	The incidence matrix of the hypergraph $\mathcal{G}^*$
N	The number of trace graphs $\mathcal{G}$/hypergraphs $\mathcal{G}^*$
N_g	The number of nodes in trace graph $\mathcal{G}$
N_h	The number of nodes in hypergraph $\mathcal{G}^*$
v_i	The i-th node in the trace graph $\mathcal{G}$
e_k	The k-th edge in the trace graph $\mathcal{G}$
n	The number of nodes in the trace graph $\mathcal{G}$
m	The number of edges in the trace graph $\mathcal{G}$
d	The dimension of nodes/edges in the trace graph $\mathcal{G}$
d'	The dimension of latent representation of nodes/edges in the trace graph $\mathcal{G}$

The center c_h and radius R_h of the hypergraph hypersphere are also obtained.

In the training phase, the train dataset only contains normal trace graphs. The total loss function is composed of the graph loss function and the hypergraph loss function:

$$\mathcal{L}_{total}(\theta, \eta) = (1 - \alpha)\mathcal{L}_{graph} + \alpha\mathcal{L}_{hypergraph}, \tag{14}$$

where $\alpha \in (0, 1)$ is hyperparameter to balance graph-view and hypergraph-view loss.

To reduce oscillations of the gradient, the momentum update is involved in optimization:

$$\theta \leftarrow optimize(\theta, \nabla_\theta \mathcal{L}_{total}(\theta, \eta), \gamma), \tag{15}$$

where γ represents the learning rate. In this context, the updates are calculated solely from the gradients concerning θ, employing the Adam [46] optimizer. Additionally, TraceHG uses the total loss function to optimize the hypergraph-view encoder in order to ensure the consistency of optimization goals.

C. Anomaly Score Computation

In the inference phase, the processed trace graphs are fed into the graph-view encoder and the hypergraph-view encoder, which generate anomaly scores, respectively. The scores are calculated as follows:

$$S_{graph}(\mathcal{G}) = ||f_{GCN}(X, A) - c_g||^2 - R_g^2, \tag{16}$$

$$S_{hypergraph}(\mathcal{G}^*) = ||f_{HyperGNN}(E, Z^*) - c_h||^2 - R_h^2. \tag{17}$$

The anomaly scores measure the shortest distance from the graph-view and hypergraph-view representation vectors to the centers of their respective hyperspheres. Normal log samples exhibit a relatively compact hypersphere, while abnormal samples are usually distributed outside the hypersphere. If either anomaly score is greater than zero, the trace is classified as an anomaly. The algorithm for the entire pipeline is presented in Algorithm 1.

Algorithm 1. Anomaly Detection Algorithm of TraceHG

Require: Raw Logs R; Batch Size B; Number of training epoch E.
Output: Anomaly detection function f_{GCN} and $f_{HyperGNN}$.
Initialize graph-view encoder parameter θ and hypergraph-view encoder parameter η.
Initialize two hypersphere centers c_g and c_h.
Initialize hyperparameters μ and α.
Extract log vector and trace graph from raw logs. Then, construct graph-view input $\mathcal{G}\{X, A\}$
Transform $\mathcal{G}$ to hypergraph-view input $\mathcal{G}^*\{E, Z^*\}$
$/ * Training\ Stage * /$
for $epoch = 1$ to $Epoch$ **do**
  $batch \leftarrow \mathcal{G}$ and $\mathcal{G}^*$ is divided with batch size by random.
  for $batch \in Batch$ **do**
    Obtain the graph-view representation $\hat{h}_n$ via (2) and (7).
    Obtain the hypergraph-view representation $\hat{h}_e$ via (8) and (12).
    Calculate the graph-view loss $\mathcal{L}_{graph}$ via (13).
    Calculate the hypergraph-view loss $\mathcal{L}_{hypergraph}$.
    Calculate the total loss $\mathcal{L}_{total}$ through $\mathcal{L}_{graph}$ and $\mathcal{L}_{hypergraph}$ via (14).
    Back propagate and update trainable parameters θ and η depends on $\mathcal{L}_{total}$.
  end for
  Update R_g and R_h after warming epochs.
end for
$/ * Inference\ Stage * /$
for $\mathcal{G}_i \in \mathcal{G}, \mathcal{G}_i^* \in \mathcal{G}^*$ **do**
  Calculate graph-view anomaly score $\mathcal{S}_{graph}$ via (16).
  Calculate hypergraph-view anomaly score $\mathcal{S}_{hypergraph}$ via (17).
end for

IV. EXPERIMENT

In this section, we conduct a series of experiments on benchmark datasets to demonstrate the effectiveness of the proposed model. Specifically, we aim to answer the following questions: **RQ1:** How efficient is the proposed method compared with SOTA baselines? **RQ2:** Can graph-view and hypergraph-view benefit each other and enhance their anomaly detection capabilities? **RQ3:** How do different components affect the performance of TraceHG? **RQ4:** How does response time affect the performance of TraceHG? **RQ5:** How is the scalability of TraceHG on the large-scale synthetic dataset? **RQ6:** How do different hyper-parameter values affect the performance of the proposed approach?

A. Experiment Settings

1) Datasets: Our research is based on three public datasets: *Train Ticket, MicroSS, and TraceBench*. The details of these datasets are outlined below.

TABLE II
THE SUMMARY OF ADOPTED BENCHMARK DATASETS

Type	Train Ticket	MicroSS	TraceBench
# of Normal trace	66,790	20,794,366	224,734
# of Abnormal trace	19,111	3,307,301	99,310
# of Inter-service anomalies	7,397	3,465	-
# of intra-service anomalies	3,692	3,298,639	-
# of Joint anomalies	8,022	5,197	-

TABLE III
THE SUMMARY OF ADOPTED BASELINES

Method	Description
DeepLog [5]	DeepLog is an LSTM-based log anomaly detection framework that detects log event level anomalies by predicting the next log event.
LogAnomaly [8]	LogAnomaly is an efficient end-to-end framework and utilizes LSTM to detect sequential and quantitative anomalies.
TraceAnomaly [51]	TraceAnomaly is an approach for anomaly detection that employs variational autoencoders (VAE) based on posterior flow to identify anomalous traces.
DeepTraLog [13]	DeepTraLog is a trace anomaly detection method that considers trace as a graph. It adopts GGNNs to learn latent representation.

Train Ticket: [47] is a medium-scale open-source microservice system based on distributed architecture. The latest release version of the Train Ticket microservice system is applied for our research. Train Ticket consists of 45 microservices utilized for accomplishing the related business of train ticket booking. The Train Ticket dataset contains 14 types of anomalies. The number of traces and each type of anomaly is shown in Table II. Based on the causes of anomalies, we categorize them into three paradigms of anomaly.

MicroSS: [48] is a large-scale microservice dataset comprising detailed trace data continuously collected over a period of two weeks, containing more than 20 million traces. This dataset is designed to simulate anomalies that frequently occur in real-world industrial systems. Although the dataset provides detailed anomaly annotations for each node within the traces, these labels are restricted to status codes (e.g., 200, 400, 500) and do not explicitly indicate the root causes. Therefore, by re-analyzing the manifestation patterns of these faults, we categorize them into three distinct paradigms. The detailed statistics regarding the number of traces and the distribution of each anomaly type are presented in Table II.

TraceBench: [49], [50] is an open-source dataset designed for trace-based monitoring, collected using MTracer on an HDFS system deployed in a real IaaS environment. It includes traces from both normal HDFS operation and scenarios with 17 injected faults, covering functional and performance issues. Over 180 hours of data collection resulted in more than 300,000 traces. The data distribution is shown in Table II. TraceBench does not provide specific causes or detailed categories for anomalies within the data, which limits our analysis to identifying anomalies without classifying them. While conducting the experiment, we reorganized all logs into trace graphs and used these to evaluate the methods.

2) Baselines: To assess the effectiveness of the proposed method, we compare TraceHG with several state-of-the-art anomaly detection approaches. Specifically, we employ four leading log-based or trace-based anomaly detection methods as baselines, as described in Table III. DeepLog and LogAnomaly are representative log-based methods, while TraceAnomaly and DeepTralog integrate both log and trace data to detect microservice anomalies.

3) Evaluation Metrics: We use several evaluation metrics, including Precision, Recall, and F1-Score, which are commonly employed in binary classification scenarios. We designate the correctly detected anomalous log sequences as True Positives (TP), the normal log sequences erroneously identified as anomalies as False Positives (FP), and the undetected anomalous log sequences as False Negatives (FN). Then we calculate the following metrics:

- *Precision:* $P = \frac{TP}{TP+FP}$
- *Recall:* $R = \frac{TP}{TP+FN}$
- *F1-score:* $F1 = \frac{2\times(Precision\times Recall)}{Precision+Recall}$

To further evaluate the comprehensiveness of anomaly detection, we add the accuracy of each type of anomaly into the evaluation metrics.

4) Experimental Setup: We implement the model using applying Pytorch [52]. All experiments are conducted on a server with 3.60 GHz Intel CPU and 31 G of RAM. The proposed model is trained to employ the Adam optimizer [46]. We apply a weight decay λ of 0.0001 and initialize the learning rate γ to 0.001. The parameter α is set to 0.5.

DeepLog and TraceAnomaly offer open-source code, enabling us to adopt them on the applied datasets. However, since DeepTraLog and LogAnomaly lack public implementations, we implement their frameworks based on their papers. Notably, both DeepLog and LogAnomaly operate at the log event level for anomaly detection. We arrange all log events chronologically and input them into the model. In the case of TraceAnomaly and DeepTraLog, we provide traces and logs as input.

B. RQ1: How Efficient is the Proposed Method Compared With SOTA Baselines?

To validate the effectiveness of TraceHG, we compared TraceHG with four state-of-the-art methods. The experiments were conducted on publicly available datasets. The results of the Train Ticket dataset, as presented in Table IV, demonstrate that the proposed approach consistently outperforms the four comparison methods in all evaluation metrics. Specifically, TraceHG achieves the highest F1-score of 96.3%, surpassing TraceAnomaly by 26.9%, DeepTraLog by 7.0%, DeepLog by 38.3%, and LogAnomaly by 34.9%. This indicates TraceHG's superior capability in identifying abnormal behaviors within system processes. Furthermore, the proposed method achieves

TABLE IV
THE COMPARISON OF FOUR STATE-OF-THE-ART MODELS ON TRACEBENCH AND TRAIN TICKET

Models	TraceBench			Train Ticket						
	Pre	*Rec*	*F1*	*Pre*	*Rec*	*F1*	*Intra-service*	*Inter-service*	*Joint*	*Statistic (P-value)*
TraceAnomaly	0.641	0.733	0.684	0.723	0.668	0.694	0.541	0.674	0.721	3.91×10^{-3}
DeepTraLog	0.941	0.856	0.896	0.905	0.881	0.893	0.967	0.901	0.823	3.91×10^{-3}
DeepLog	0.676	**0.995**	0.805	0.561	0.599	0.580	0.524	0.761	0.485	7.81×10^{-3}
LogAnomaly	0.745	0.923	0.825	0.726	0.532	0.614	0.567	0.637	0.419	3.91×10^{-3}
TraceHG-SB	0.899	0.908	0.903	0.951	0.914	0.932	0.871	0.905	0.942	7.81×10^{-3}
TraceHG-OCSVM	0.914	0.769	0.835	0.875	0.656	0.750	0.746	0.558	0.705	3.91×10^{-3}
TraceHG-GRU	0.823	0.904	0.862	0.660	0.750	0.746	0.973	0.455	0.823	3.91×10^{-3}
TraceHG-LSTM	0.795	0.846	0.820	0.765	0.651	0.703	0.813	0.561	0.659	3.91×10^{-3}
TraceHG-GGNN	0.923	0.870	0.896	0.956	**0.943**	0.949	0.906	0.934	**0.967**	6.87×10^{-2}
TraceHG	**0.944**	0.946	**0.945**	**0.984**	0.942	**0.963**	**0.996**	**0.935**	0.926	-

The best result is highlighted in bold font. The second-best result is marked with underlining for clarity.

TABLE V
THE COMPARISON OF FOUR STATE-OF-THE-ART MODELS ON MICROSS

Models	*Pre*	*Rec*	*F1*	*Intra-service*	*Inter-service*	*Joint*
TraceAnomaly	0.520	0.650	0.578	0.650	0.432	0.557
DeepTraLog	0.943	0.924	0.933	0.924	0.948	0.937
DeepLog	0.673	0.905	0.772	0.905	0.696	0.863
LogAnomaly	0.723	**0.991**	0.836	**0.992**	0.431	0.793
TraceHG	**0.952**	0.931	**0.941**	0.931	**0.967**	**0.938**

The best result is highlighted in bold font. The second-best result is marked with underlining for clarity.

the highest recall of 94.2% compared to the other four baselines, showing a significant improvement over TraceAnomaly by 27.4%, DeepTraLog by 6.1%, DeepLog by 34.3%, and LogAnomaly by 41.0%. These results suggest that TraceHG consistently outperforms the four comparison methods across all evaluated metrics.

Furthermore, we conducted experiments on the TraceBench dataset. TraceHG shows significantly better performance than the other approaches. As shown in Table IV, TraceHG achieves the best F1-score of 94.5%, outperforming TraceAnomaly by 26.1%, DeepTraLog by 4.9%, DeepLog by 14.0%, and LogAnomaly by 12.0%. This highlights the superiority of TraceHG in the microservice anomaly detection task. To be specific, TraceHG achieves notable precision gains of 30.3%, 0.03%, 26.8%, and 19.9% over TraceAnomaly, DeepTraLog, DeepLog, and LogAnomaly, respectively.

It is important to note that while DeepLog demonstrates the highest recall among all methods in TraceBench, its precision is significantly lower compared to other approaches. This can be attributed to the fact that DeepLog exhibits a tendency to classify unknown samples as anomalies, leading to a high false positive rate. Therefore, in terms of overall detection performance, TraceHG remains the superior approach.

Moreover, we conducted experiments on the large-scale MicroSS dataset. As presented in Table V, the proposed TraceHG demonstrates superior performance, achieving the highest F1-score of 94.1% and a precision of 95.2%. Specifically, in terms of F1-score, TraceHG outperforms TraceAnomaly by 36.3%, DeepTraLog by 0.8%, DeepLog by 16.9%, and LogAnomaly by 10.5%. Similarly, TraceHG achieves notable precision gains of 43.2%, 0.9%, 27.9%, and 22.9% over TraceAnomaly, DeepTraLog, DeepLog, and LogAnomaly, respectively. It is worth noting that while LogAnomaly achieves the highest recall, its precision is considerably lower compared to TraceHG. Consistent with observations in TraceBench, log-based anomaly detection methods, such as LogAnomaly and DeepLog, exhibit a tendency to classify unseen or rare samples as anomalies. Consequently, TraceHG achieves a better trade-off between precision and recall, remaining the superior approach in terms of overall detection performance.

To further verify TraceHG's comprehensiveness of anomaly detection, we investigated the recall of three types of anomalies in the Train Ticket and MicroSS datasets. In comparison to the most competitive baselines, TraceHG demonstrates the comprehensive capability of anomaly detection. Specifically, in terms of joint anomalies, TraceHG achieves the best recall of 92.6%, outperforming TraceAnomaly by 20.5%, DeepTraLog by 10.3%, DeepLog by 44.1%, and LogAnomaly by 50.7%. Similarly, on the MicroSS dataset, TraceHG achieves superior performance in both inter-service and joint anomalies. Specifically, regarding inter-service anomalies, TraceHG reaches 96.7% recall, surpassing TraceAnomaly, DeepTraLog, DeepLog, and LogAnomaly by 53.5%, 1.9%, 27.1%, and 53.6%, respectively. For joint anomalies, TraceHG achieves 93.8% recall, outperforming the aforementioned baselines by 38.1%, 0.1%, 7.5%, and 14.5%, respectively.

It is important to note that since the quantity of intra-service anomalies far exceeds that of other categories in the MicroSS dataset, the overall recall is predominantly determined by this type of anomaly. Consequently, although some baselines (e.g., LogAnomaly) achieve high overall recall, they fail to capture complex inter-service and joint anomalies effectively. Overall, we observe that log-based methods show poorer performance

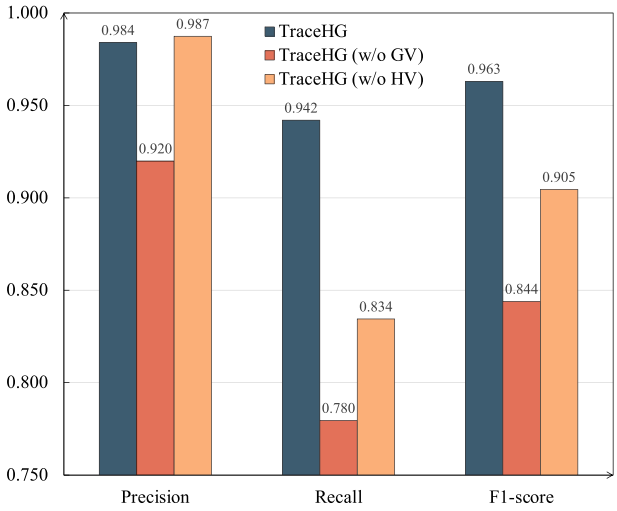

Fig. 6. Performance comparison of different models on Train Ticket dataset.

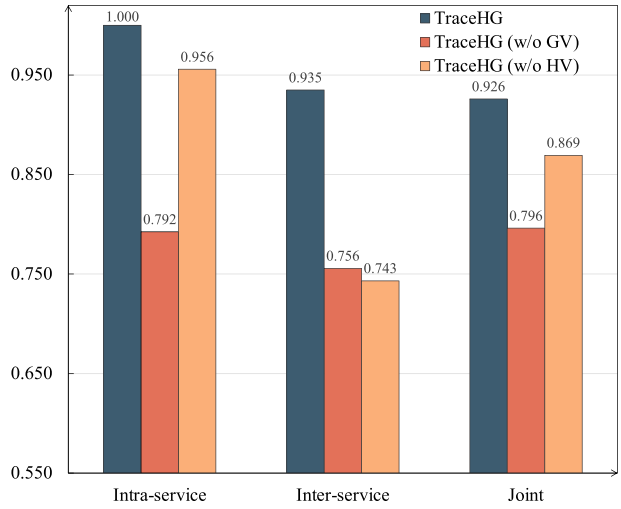

Fig. 7. Performance comparison for different types of anomalies on Train Ticket dataset.

than trace-based approaches in these complex scenarios. This indicates that trace-based methods can detect microservice anomalies more efficiently by mining more structured information.

Apart from these evaluation metrics, the Wilcoxon signed-ranks test [53], a statistical strategy, was introduced to further analyze the experimental results. The Wilcoxon signed-ranks test is one of the simple yet effective non-parametric testing methods. It evaluates the significance of the proposed TraceHG compared to the other methods. The p-values representing the significance are shown in Table IV. It can be observed that the p-values of the four baselines (TraceAnomaly, DeepTraLog, DeepLog, and LogAnomaly) are lower than 0.05. These results demonstrate that TraceHG significantly outperforms the other baseline methods at a significance level of 0.05.

Nevertheless, the absence of specific anomaly categories in the TraceBench dataset limited our ability to evaluate the performance of TraceHG in identifying each type of anomaly. Despite this limitation, we can still observe the remarkable performance of TraceHG through the overall classification metrics.

C. RQ2: Can Graph-View and Hypergraph-View Benefit Each Other and Enhance Their Anomaly Detection Capabilities?

To delve deeper into the impact of both the graph view and hypergraph view on TraceHG, we conducted an ablation study using the Train Ticket dataset. For clarity, **TraceHG (w/o GV)** and **TraceHG (w/o HV)** denote scenarios where only the hypergraph view or the graph view is utilized, respectively. TraceHG indicates all components are available. As depicted in Fig. 6, we observe that TraceHG achieves the highest F1-score at 96.3%, outperforming TraceHG (w/o GV) by 11.9% and TraceHG (w/o HV) by 14.8%. This underscores the indispensability of both the graph view and hypergraph view in anomaly detection.

Meanwhile, we investigated the effect of the two views on the accuracy of different anomaly types. As shown in Fig. 7, it is evident that the absence of either view significantly impacts the accuracy of detecting the three types of anomalies. This suggests that the combination of the graph view and hypergraph view captures more associations between microservices. It is crucial to adopt both views for detection simultaneously. Leveraging both views synergistically yields optimal anomaly detection results.

D. RQ3: How Do Different Components Affect the Performance of TraceHG?

To assess the contributions of different components to the proposed model, we performed an ablation study by replacing various components in TraceHG and evaluating the impact on performance.

1) Anomaly Score Strategy: We employed distinct anomaly scoring strategies to determine abnormal traces while maintaining stability in both the GNN encoder and HGNN encoder. We introduced two variants of TraceHG: **TraceHG-SB** and **TraceHG-OCSVM**. TraceHG-SB utilizes a soft boundary approach [45] instead of a conventional one-class decision boundary, while TraceHG-OCSVM [54] incorporates one-class SVM for anomaly detection instead of DeepSVDD. Analysis from Table IV reveals that TraceHG outperforms the other variants. These findings underscore the efficacy of the anomaly-scoring strategies.

2) Encoder Model: We trained different encoder models instead of the original one. In order to ensure that the dual-view framework remained valid, only the graph-view encoder was substituted. We introduced three TraceHG variants: **TraceHG-GRU**, **TraceHG-LSTM**, and **TraceHG-GGNN**, where the GCN was replaced by GRU [55], LSTM [56], and GGNN [57], respectively. Results are presented in Table IV. Across both datasets, TraceHG consistently achieves either the best or the second-best performance on all evaluation metrics, underscoring the effectiveness of incorporating GCN in its design.

E. RQ4: How Does Response Time Affect the Performance of TraceHG?

To quantify the specific contribution of response time, we conducted an ablation study by replacing response time with a

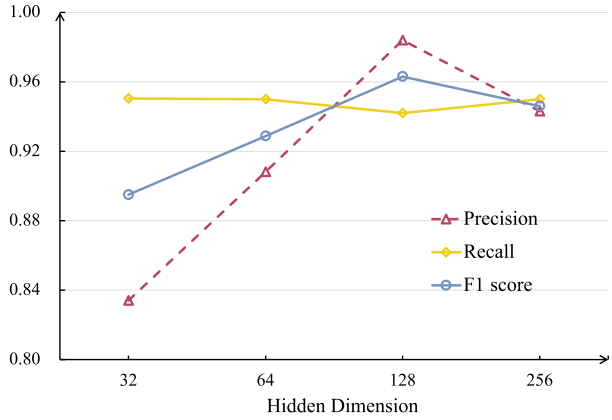

Fig. 8. Performance comparison with different hidden dimensions of TraceHG on Train Ticket dataset.

TABLE VI
ABLATION STUDY OF RESPONSE TIME

Models	*Pre*	*Rec*	*F1*	*Intra-service*	*Inter-service*	*Joint*
TraceHG ($RT = 1$)	0.927	0.931	0.929	0.986	0.914	0.921
TraceHG ($RT = 0$)	0.932	0.915	0.923	0.955	0.900	0.910
TraceHG	**0.984**	**0.942**	**0.963**	**0.996**	**0.935**	**0.926**

constant value, denoted as TraceHG with $RT = 0$ and $RT = 1$. For the graph view, we replaced edge weights with constants to force the model to rely solely on topological dependencies. Crucially, for the hypergraph view, simply removing unique time values would have caused structural collapse. Therefore, we preserved the original incidence matrix constructed from the raw trace but masked the input node features with constants. This strategy effectively eliminated response time while maintaining the structural integrity of the hypergraph topology.

As presented in Table VI, TraceHG with response time consistently exhibits superior performance in both ablation settings. Specifically, in terms of F1-score, TraceHG outperforms the variants with $RT = 1$ and $RT = 0$ by 3.4% and 4.0%, respectively. This advantage is particularly pronounced in the recall of inter-service and joint anomalies. For inter-service anomalies, TraceHG surpasses the $RT = 1$ and $RT = 0$ settings by 2.1% and 3.5%, respectively. Similarly, for joint anomalies, it achieves gains of 0.5% and 1.6%, respectively. These results empirically verify that response time plays a pivotal role in distinguishing inter-service and joint anomalies.

F. RQ5: How is the Scalability of TraceHG on Large-Scale

To investigate the scalability of TraceHG, we constructed synthetic datasets by expanding the scale of traces based on the Train Ticket dataset. To ensure that the original topological relationships within the trace were preserved, we first identified the invocation dependencies and then randomly performed cyclic duplication of entire call branches. We generated two distinct settings where traces were expanded to exceed 100 and 300 nodes, respectively.

As shown in Table VII, when the trace scale increases to 100 and 300 nodes, TraceHG exhibits a slight decline due to the increased complexity. However, TraceHG still maintains an F1-score above 80% (specifically, 83.3% in the 300-node setting), demonstrating its robust capability in identifying anomalies within large-scale microservice traces.

TABLE VII
PERFORMANCE OF TRACEHG ON SYNTHETIC DATASETS WITH VARYING SCALES

Setting	*Pre*	*Rec*	*F1*	*Intra-service*	*Inter-service*	*Joint*
Train Ticket-300	0.891	0.783	0.833	0.821	0.831	0.721
Train Ticket-100	0.922	0.856	0.888	0.832	0.862	0.861
Train Ticket	**0.984**	**0.942**	**0.963**	**0.996**	**0.935**	**0.926**

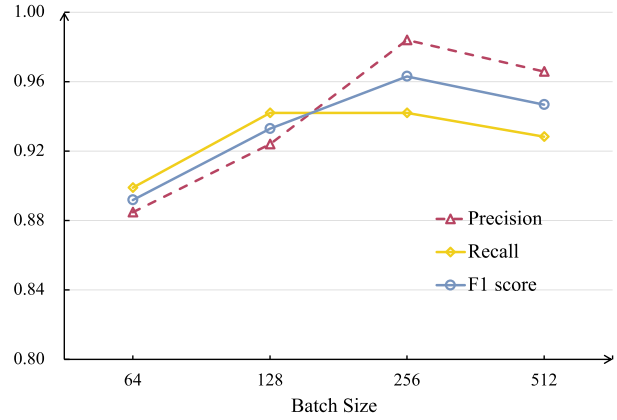

Fig. 9. Performance comparison with different batch sizes of TraceHG on Train Ticket dataset.

G. RQ6: How Do Different Hyper-Parameter Values Affect the Performance of the Proposed Approach?

In this section, we conducted a series of experiments on the impact of different hyper-parameters on the proposed model. We compared TraceHG with different *Hidden Dimensions* and *Batch Sizes* on the Train Ticket dataset.

1) Hidden Dimension: To investigate the impact of hidden dimensions on TraceHG, we varied the hidden dimension of the graph encoder within the range of 32, 64, 128, 256 and evaluated the corresponding precision, recall, and F1-score. The outcomes are depicted in Fig. 8. Analysis of Fig. 8 reveals that the effectiveness of the proposed method steadily enhances with an increase in hidden dimension within the range of [32, 128]. Nevertheless, the rate of performance improvement diminishes as the hidden dimension exceeds 128. Consequently, we opted to set the hidden dimension at 128.

2) Batch Size: In order to explore the influence of the selection of batch size, we experimented with values ranging from 64, 128, 256, 512 and analyzed the corresponding metrics. The results are consolidated in Fig. 9. Broadly, we observe a smooth enhancement in performance as the batch size escalates from 64 to 256. However, beyond this point, augmenting the batch size yields negligible improvements in performance. Consequently, we settled on a batch size of 256. It is noteworthy that the selection of batch size impacts the construction of the hypersphere. The optimal batch size configuration heavily depends on the

dataset's characteristics, often determined by the prioritization between precision and recall.

V. Discussion

In this section, we discuss and compare single-view and dual-view anomaly detection, and different log processing manners.

A. Single-View Vs. Dual-View

The introduction of dual-view anomaly detection has demonstrated unparalleled performance compared to single-view and other methods. In the proposed TraceHG, the original trace graph is constituted by log events and time intervals, which map to nodes and edges, respectively. Single-view graph encoder learns graph-level representation which reflects semantic information of logs and invocations between microservices. Correspondingly, in the hypergraph, the nodes and hyperedges are mapped from the edges and nodes of the original trace graph. On the one hand, graph-view anomaly detection focuses on mining information from nodes and only implicitly captures edge information. It is merely proficient in detecting individual microservice or partial invocation anomalies. On the other hand, hypergraph view could capture invocation and topology relationships between microservices. Moreover, it supplements the absence of explicit response time anomaly detection in traces and perceives abundant information from microservice logs. Hence, the incorporation of hypergraph-view anomaly detection enhances the capability of detecting response time and joint anomalies.

B. Topological Vs. Sequential Microservice Trace

The proposed TraceHG treats trace and log as topology data. On the contrary, other approaches process logs in time sequential order. Trace reflects calls and interactions between microservices. Unilateral and simple sequential log processing ignores the asynchronous or parallel calling relationships between microservices. For example, when a microservice calls two sub-microservices, one exhibits swift response times while the other responds slowly. In the time dimension, the promptly responding service holds greater relevance to the parent service. Nonetheless, as both are sub-microservices of the parent service, they should maintain equal importance in terms of their invocation relationships. Utilizing topological relationship extraction can retain crucial the characteristics of microservice and alleviate this issue. In the topological structure, these two sub-microservices are assigned equal importance. Simultaneously, the proposed method incorporates response time reasonably into the topology graph, ensuring it does not cause false negatives in detecting response time anomalies.

VI. Conclusion

In this paper, we proposed a novel unsupervised dual-view method named TraceHG. In TraceHG, we incorporated the response time of traces into our framework to effectively identify inter-service, intra-service, and joint anomalies. Furthermore, a graph view and a hypergraph view were adopted simultaneously to mine the hidden representations of microservice traces. Two hyperspheres were constructed for each view, and anomaly scores were computed accordingly. Across two trace datasets from microservice systems, the proposed TraceHG consistently outperforms all baselines, achieving outstanding F1-scores. Notably, the proposed approach is capable of dealing with all types of anomalies. The results on the ablation study demonstrate the effectiveness of the model design and the synergistic combination of the graph view and the hypergraph view.

Limitations and Future Work: Although TraceHG achieves comprehensive anomaly detection, it lacks a significant advantage in terms of detection time due to the integration of both graph view and hypergraph view branches. In the future, we will continue to explore holistic microservice anomaly detection solutions that balance detection efficiency and accuracy. Furthermore, we aim to extend TraceHG to accommodate a variety of application scenarios within microservice systems. Additionally, we plan to evaluate TraceHG across a broader spectrum of microservice system architectures.

References

[1] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Microservices architecture enables DevOps: Migration to a cloud-native architecture," *IEEE Softw.*, vol. 33, no. 3, pp. 42–52, May/Jun. 2016.

[2] T. Erl, *Service-Oriented Architecture: Concepts, Technology, and Design*. Noida, India: Pearson Education India, 1900.

[3] T. Cerny, M. J. Donahoo, and M. Trnka, "Contextual understanding of microservice architecture: Current and future directions," *ACM SIGAPP Appl. Comput. Rev.*, vol. 17, no. 4, pp. 29–45, 2018.

[4] N. Han, S. Lu, D. Wang, M. Wang, X. Tan, and X. Wei, "SKDLog: Self-knowledge distillation-based CNN for abnormal log detection," in *Proc. IEEE Smartworld, Ubiquitous Intell. Comput., Scalable Comput. Commun., Digi. Twin, Privacy Comput., Metaverse, Auton. Trusted Veh.*, 2022, pp. 796–805.

[5] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.

[6] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *Proc. IEEE/ACM 45th Int Conf. Softw. Eng. (ICSE)*, 2023, pp. 1750–1762.

[7] S. Nedelkoski, J. Cardoso, and O. Kao, "Anomaly detection from system tracing data using multimodal deep learning," in *Proc. IEEE 12th Int. Conf. Cloud Comput.*, 2019, pp. 179–186.

[8] W. Meng et al., "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 4739–4745.

[9] R. Chen et al., "LogTransfer: Cross-system log anomaly detection for software systems with transfer learning," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng.*, 2020, pp. 37–47.

[10] Z. Xie et al., "Unsupervised anomaly detection on microservice traces through graph VAE," in *Proc. ACM Web Conf.*, 2023, pp. 2874–2884.

[11] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, 2020, pp. 1–9.

[12] A. Nandi, A. Mandal, S. Atreja, G. B. Dasgupta, and S. Bhattacharya, "Anomaly detection using program control flow graph mining from execution logs," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 215–224.

[13] C. Zhang et al., "DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 623–634.

[14] J. Chen et al., "TraceGra: A trace-based anomaly detection for microservice using graph deep learning," *Comput. Commun.*, vol. 204, pp. 109–117, 2023.

[15] X. Zhou et al., "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, Feb. 2021.

[16] R. Tighilt et al., "On the study of microservices antipatterns: A catalog proposal," in *Proc. Eur. Conf. Pattern Lang. Programs*, 2020, pp. 1–13.
[17] Y. Song, R. Xin, P. Chen, R. Zhang, J. Chen, and Z. Zhao, "Autonomous selection of the fault classification models for diagnosing microservice applications," *Future Gener. Comput. Syst.*, vol. 153, pp. 326–339, 2024.
[18] A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, and M. Kocaoglu, "Root cause analysis of failures in microservices through causal discovery," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 31158–31170.
[19] S. Lu, X. Wei, Y. Li, and L. Wang, "Detecting anomaly in Big Data system logs using convolutional neural network," in *Proc. IEEE 16th Int. Conf. Dependable, Autonomic Secure Comput., 16th Int. Conf. Pervasive Intell. Comput., 4th Int. Conf. Big Data Intell. Comput. Cyber Sci. Technol. Congr.*, 2018, pp. 151–158.
[20] A. Farzad and T. A. Gulliver, "Log message anomaly detection with fuzzy C-means and MLP," *Appl. Intell.*, vol. 52, no. 15, pp. 17708–17717, 2022.
[21] S. Ying et al., "An improved KNN-based efficient log anomaly detection method with automatically labeled samples," *ACM Trans. Knowl. Discov. Data*, vol. 15, no. 3, pp. 1–22, 2021.
[22] S. Lu, N. Han, M. Wang, X. Wei, Z. Lin, and D. Wang, "SSDLog: A semi-supervised dual branch model for log anomaly detection," *World Wide Web*, vol. 26, no. 5, pp. 3137–3153, 2023.
[23] S. Lu et al., "Black-box attacks against log anomaly detection with adversarial examples," *Inf. Sci.*, vol. 619, pp. 249–262, 2023.
[24] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," in *Proc. IEEE 2021 Int. Joint Conf. Neural Netw.*, 2021, pp. 1–8.
[25] H. Uchida, K. Tominaga, H. Itai, Y. Li, and Y. Nakatoh, "Multi-parameter log anomaly detection with an unsupervised learning approach," in *Proc. 2024 Int. Symp. Parallel Comput. Distrib. Syst.*, 2024, pp. 1–5.
[26] L. Zhang, T. Jia, M. Jia, Y. Li, Y. Yang, and Z. Wu, "Multivariate log-based anomaly detection for distributed database," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 4256–4267.
[27] G. Yu, Z. Huang, and P. Chen, "TraceRank: Abnormal service localization with dis-aggregated end-to-end tracing data in cloud native systems," *J. Softw., Evol. Process*, vol. 35, no. 10, 2023, Art. no. e2413.
[28] C. Zhang et al., "TraceCRL: Contrastive representation learning for microservice trace analysis," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2022, pp. 1221–1232.
[29] J. Bogatinovski, S. Nedelkoski, J. Cardoso, and O. Kao, "Self-supervised anomaly detection from distributed traces," in *Proc. IEEE/ACM 13th Int. Conf. Utility Cloud Comput.*, 2020, pp. 342–347.
[30] K. Aktaş and H. H. Kilinc, "Interaction prediction and anomaly detection in a microservices-based telecommunication platform," in *Proc. 2024 Int. Conf. Softw. Syst. Processes*, 2024, pp. 56–65.
[31] L. Meng, F. Ji, Y. Sun, and T. Wang, "Detecting anomalies in microservices with execution trace comparison," *Future Gener. Comput. Syst.*, vol. 116, pp. 291–301, 2021.
[32] M. Li, D. Tang, Z. Wen, and Y. Cheng, "Microservice anomaly detection based on tracing data using semi-supervised learning," in *Proc. 4th Int. Conf. Artif. Intell. Big Data*, 2021, pp. 38–44.
[33] K. Shi, J. Li, Y. Liu, Y. Chang, and X. Li, "BSDG: Anomaly detection of microservice trace based on dual graph convolutional neural network," in *Proc. Int. Conf. Serv.-Oriented Comput.*, 2022, pp. 171–185.
[34] H. Liu et al., "UAC-AD: Unsupervised adversarial contrastive learning for anomaly detection on multi-modal data in microservice systems," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3887–3900, Nov./Dec. 2024.
[35] Z. Li, J. Zhao, and J. Kang, "Multi-source anomaly detection for microservice systems," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng., Companion Proc.*, 2024, pp. 414–415.
[36] P. Wang, X. Zhang, Y. Chen, and Z. Cao, "Unsupervised microservice system anomaly detection via contrastive multi-modal representation clustering," *Inf. Process. Manage.*, vol. 62, no. 3, 2025, Art. no. 104013.
[37] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Serv.*, 2017, pp. 33–40.
[38] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2014, pp. 1532–1543.
[39] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.
[40] Y. Li, N. Du, and S. Bengio, "Time-dependent representation for neural event sequence prediction," 2017, *arXiv:1708.00065*.
[41] E. R. Scheinerman and D. H. Ullman, *Fractional Graph Theory: A Rational Approach to the Theory of Graphs*. North Chelmsford, MA, USA: Courier Corporation, 2011.
[42] J. Jo, J. Baek, S. Lee, D. Kim, M. Kang, and S. J. Hwang, "Edge representation learning with hypergraphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 7534–7546.
[43] L. Yao, C. Mao, and Y. Luo, "Graph convolutional networks for text classification," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 7370–7377.
[44] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 3558–3565.
[45] L. Ruff et al., "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
[46] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[47] X. Zhou et al., "Latent error prediction and fault localization for microservice applications by learning from system trace logs," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2019, pp. 683–694.
[48] FlyFish, "Gaia," 2024. [Online]. Available: http://docs.aiops.cloudwise.com/en/gaia/
[49] J. Zhou, Z. Chen, J. Wang, Z. Zheng, and M. R. Lyu, "Trace bench: An open data set for trace-oriented monitoring," in *Proc. IEEE 6th Int. Conf. Cloud Comput. Technol. Sci.*, 2014, pp. 519–526.
[50] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng.*, 2023, pp. 355–366.
[51] P. Liu et al., "Unsupervised detection of microservice trace anomalies through service-level deep Bayesian networks," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng.*, 2020, pp. 48–58.
[52] A. Paszke et al., "Automatic differentiation in PyTorch," *31st Conf. Neural Inf. Process. Syst. (NIPS 2017)*, Long Beach, CA, USA, 2017.
[53] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.
[54] L. M. Manevitz and M. Yousef, "One-class SVMs for document classification," *J. Mach. Learn. Res.*, vol. 2, pp. 139–154, 2001.
[55] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1724–1734.
[56] A. Graves and A. Graves, "Long short-term memory," *Supervised Sequence Labelling Recurrent Neural Netw.*, 2012, pp. 37–45.
[57] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, "Gated graph sequence neural networks," 2015, *arXiv:1511.05493*.

Ningning Han (Member, IEEE) received the BS degree in software engineering from Yanshan University, Qinhuangdao, China, in 2022. She is currently working toward the MS degree in cyberspace security with the School of Computer Science and Technology, Beijing Jiaotong University, Beijing, China. She is also a visiting student with the University of Electro-Communications, Tokyo, Japan. Her research interests include deep learning, data mining, and anomaly detection.

Siyang Lu (Member, IEEE) received the PhD degree in computer science from University of Central Florida in 2019. He is currently an associate professor with the School of Computer Science and Technology, Beijing Jiaotong University. His research interests include anomaly detection and deep learning. He has held one search grant from the National Science Foundation of China (NSFC).

Zaichao Lin (Graduate Student Member, IEEE) received the BS degree in digital media technology from North China University of Technology, Beijing, China, in 2023. He is currently working toward the postgraduation degree with Beijing Jiaotong University, Beijing. His research interests include information security and artificial intelligence.

Bin Li (Member, IEEE) received the BS degree in software engineering from Hebei University, Baoding, China, in 2024. He is currently working toward the postgraduation degree with Beijing Jiaotong University, Beijing, China. His research interests include information security and artificial intelligence.

Nan Wang (Graduate Student Member, IEEE) received the BE degree from the Harbin Institute of Technology, Weihai, China, in 2016, and the PhD degree from Tsinghua University, Beijing, China, in 2021. She is currently an assistant professor with the School of Cyberspace Science and Techonology, Beijing Jiaotong University, Beijing.

Xin Luo (Fellow, IEEE) received the B.S. degree in computer science from the University of Electronic Science and Technology of China, Chengdu, China, in 2005, and the Ph.D. degree in computer science from the Beihang University, Beijing, China, in 2011. He is currently a Distinguished Professor of Data Science and Computational Intelligence, and serving as the Dean of the College of Computer and Information Science, and School of Software, Southwest University, Chongqing, China. He has authored or coauthored over 400 papers (including over 190 IEEE Transactions/Journal papers) in the areas of Artificial Intelligence and Data Science, receiving 22,000+ Google Scholar citations with the H-Index of 87. Dr. Luo was the recipient of the Outstanding Associate Editor Award from IEEE Access in 2018, IEEE/CAA Journal of Automatica Sinica in 2020, and from IEEE Transactions on Neural Networks and Learning Systems in 2022-2024. He is currently serving as an Associate Editor for IEEE Transactions on Neural Networks and Learning Systems, and IEEE/CAA Journal of Automatica Sinica. His Google Scholar page is given at the link https://scholar.google.com/citations?user=hyGlDs4AAAAJ&hl=zh-CN.